

Smart Parking AI System

Manual Polygon Setup & Universal ML Pipeline

Prepared by: Antigravity AI Assistant

Architecture: YOLO Universal Car Detection + Static Geometry

1. The Core Architecture

Instead of training an AI to understand 'empty' vs 'occupied' parking patches, we separate the system into two distinct parts:

- 1. Static Geometry:** The physical parking slots mapped as polygons.
- 2. AI Vehicle Detection:** A YOLO model that only detects 'Cars'.

2. Deep Dive: The Universal ML Strategy

Because our architecture relies on the 'Center-Point Polygon' logic, the Machine Learning goal is extremely straightforward: **Detect every vehicle in the frame, regardless of environmental conditions.** We are building a single 'Universal Model' that acts as an indestructible vehicle detector.

Step A: Dataset Curation & Transfer Learning

A neural network is only as smart as the diversity of its data. Because we are starting with a pre-trained COCO model (which excels at street-level views), we must enhance it with datasets that provide overhead variety and weather robustness:

Dataset	What it teaches the AI	Conditions Provided
CARPK Dataset	The gold standard for parking. 90k+ vehicles.	Drones/High-angle camera views
UA-DETRAC	Extreme weather and lighting robustness.	Rain, night, deep shadows, glare
VisDrone	Separating tightly packed vehicles.	Extreme top-down perspectives

Step B: Data Standardization (The Class 0 Merge)

Since we are pulling from 3 different datasets, they all have different labeling structures. Some label cars as 'Class 3', some as 'Class 1', and some include buses and trucks. We will write a **Python Parsing Pipeline** that processes 50,000+ `.txt` label files and rewrites them into the standard YOLO format:

[Class_ID] [X_Center] [Y_Center] [Width] [Height]

Crucially, our script will force every single vehicle type (car, truck, van, bus) to become a single class: **Class 0 (Vehicle)**. This simplifies the YOLO model's job, as it no longer wastes compute trying to distinguish a truck from a car; it just knows 'this object occupies a parking spot'.

Step C: The Training Phase & Hyperparameters

We utilize **YOLOv8n (Nano)** because it provides real-time processing speeds (>40 FPS on edge devices) with minimal accuracy loss compared to heavier models. We initialize it with pre-trained COCO weights (Transfer Learning), giving the AI a head start.

Training Configuration (hyp.yaml):

- epochs: 150 (To allow the model to fully digest the massive composite dataset).
- batch: 32 (Maximizing GPU VRAM usage).
- imgsz: 640 (Standardizing input resolution for speed).
- mosaic: 1.0 (YOLO's Mosaic augmentation creates composite images, drastically improving detection of tightly packed, partially obscured cars).

Step D: Recall-Optimized Validation

In standard ML, you optimize for an even balance of Precision and Recall (F1 Score). For parking, we intentionally tune the model to heavily favor **Recall**.

Why? If the model has a 'False Positive' (it thinks a shopping cart is a car), our Geometry Engine filters it out (because the shopping cart isn't parked neatly in the center of a slot polygon). However, if the model has a 'False Negative' (it completely misses a black car at night), the system will tell a user the spot is empty, leading to frustration. Therefore, we lower the Confidence Threshold (e.g., conf=0.25) to ensure the AI spots every single vehicle.

3. Backend Architecture (FastAPI & Core Logic)

The Backend is the high-speed engine of the system, written in Python using FastAPI. It bridges the AI inference with the user interface.

1. Inference & Geometry Engine:

A background worker continuously reads the RTSP camera stream and passes frames to the YOLO model. It then calculates the 'Bottom-Center' point of every detected car. Using `cv2.pointPolygonTest()`, it checks these points against the JSON polygon configurations to determine slot occupancy.

2. WebSocket Broadcasting:

Every time a car enters or leaves a polygon, the backend instantly pushes a lightweight JSON payload (e.g., { A1: 'Occupied', B2: 'Empty' }) to the frontend via WebSockets.

3. Database Management (PostgreSQL):

State changes are logged asynchronously to a PostgreSQL database. This allows for historical analytics (e.g., calculating the average parking duration of a specific slot over a month).

4. Frontend Architecture (React / Next.js)

The Frontend provides a premium, responsive experience for both System Administrators and End Users, utilizing Next.js and TailwindCSS.

The Admin 'Setup' Portal:

- **Canvas Mapping Tool:** Instead of a raw Python OpenCV script, Admins use a web-based HTML5 Canvas tool. They upload a camera reference frame, click to draw polygons, and name the slots directly in the browser.

The Live Dashboard (For Users/Managers):

- **Real-Time Interactive Map:** Subscribes to the WebSocket. Polygons dynamically shift between Red and Green.
- **Analytics & Metrics:** Beautiful charts (using Recharts or Chart.js) displaying peak occupancy hours, available spaces, and estimated revenue.

5. Edge-Case Handling & System Robustness

To elevate the system from a prototype to an enterprise-grade solution, we must handle real-world edge cases like bad drivers and camera flickering.

A. Temporal Smoothing (Debouncing) to Prevent 'Flicker':

If a car drives down the aisle and its bounding box temporarily clips an empty parking spot, a naive system would instantly flash that spot as 'Occupied' for half a second. To solve this 'Drive-by Flicker', we implement a state-machine delay: **A car must be continuously detected inside the polygon for 3 consecutive seconds (e.g., 90 frames)** before the database registers a state change. This completely eliminates noise from passing vehicles or transient camera glitches.

B. The 3-out-of-5 Point Geometric Verification:

Relying solely on the 'Center-Point' of a car is fragile. If a driver parks terribly (e.g., diagonally across a line), the center point might sit in the aisle. Instead, we extract 5 points from the YOLO bounding box: Top-Left, Top-Right, Bottom-Left, Bottom-Right, and Center. We iterate over these points using `cv2.pointPolygonTest()`. **If at least 3 of these 5 points fall inside the polygon, the spot is Occupied.** Because of rectangle geometry, requiring 3 points mathematically forces the system to verify that the vast majority of the vehicle's physical mass is inside the spot.

6. User Wayfinding & Interactive Navigation

A driver in a massive parking lot does not know where 'Slot A12' is located. To create a premium user experience, we translate raw occupancy data into actionable directions.

A. The In-App Dynamic Routing (A* Algorithm):

The User Dashboard displays a 2D interactive map of the parking lot. The driving aisles are mapped with hidden 'waypoints' (nodes). When the user clicks '**Find Me a Spot**', the frontend runs a routing algorithm (like A* Search) to find the shortest path from the Entrance to the nearest Available (Green) polygon. It instantly draws a glowing blue line connecting the waypoints, guiding the driver turn-by-turn directly into the empty slot.

B. Live Rerouting (Chaos Handling):

Because the backend streams data via WebSockets, the frontend is always aware of the live state. If the user is driving towards Slot A12, but another car suddenly takes A12 before they arrive, the WebSocket instantly updates `liveState['A12'] = Occupied`. The routing algorithm detects this collision, deletes the old route, and instantly calculates a new blue line to the next available slot.

7. Executive Summary: The Final MVP Flow

This is the end-to-end data journey of the final architecture, from the camera lens to the user's phone.

Phase 1: The ML Preparation (Done Once)

- Combine images from CARPK, VisDrone, and UA-DETRAC.
- Convert all labels to Class 0: Vehicle (ignoring empty spaces).
- Run Transfer Learning using `yolov8n.pt` (COCO) as the base, training for 150 epochs to create an indestructible vehicle detector: `ultimate\_parking\_model.pt`.

Phase 2: Lot Configuration (Done Once Per Camera)

- A new camera is installed in the parking lot.
- The Admin takes a snapshot of the live feed and uploads it to the Web Polygon Tool.
- The Admin draws polygons around the painted spots and names them (A1, A2, etc).
- The tool exports a slots_config.json file.

Phase 3: The Live Backend Engine (Runs 30x per second)

- **Inference:** Grab a frame from the live CCTV. YOLO outputs the bounding box `[x1, y1, x2, y2]` for every car.
- **The 5-Point Check:** The Geometry Engine extracts 5 points (Center + 4 Corners).
- **Intersection Math:** If at least 3 out of 5 points fall inside Polygon A1, the car is physically inside the spot.
- **Temporal Smoothing:** The backend holds this state for 3 continuous seconds to ignore drive-bys.
- **Database Update:** Once 3 seconds pass, the database logs: Status[A1] = Occupied.

Phase 4: The Real-Time User Experience (Instantaneous)

- The millisecond the database updates, the FastAPI backend fires a message over WebSockets to the Frontend Web App.
- **The 2D Map:** The interactive schematic map instantly flips Polygon A1 from Green to Red.
- **Smart Routing:** If a driver was currently following the blue navigation line to Slot A1, the app detects the collision, deletes the blue line, and instantly reroutes them to Slot A2 using the A* algorithm.